

Architetture e Sistemi Operativi

Contents

1	Introduzione	3
2	Processore, memoria e istruzioni	4
2.1	Processore	4
2.2	Memoria	4
2.3	Tipi di istruzioni	4

1 Introduzione

In questo corso andremo a:

- Capire come funziona un elaboratore.
- Capire quali tipi di istruzioni è in grado di eseguire un elaboratore nativamente.
- Come lo si può dotare di un sistema operativo.

Il corso utilizza Windows e Linux come esempi di sistemi operativi.

Codice Teams: 6uk5ntg

Email Docente: marco.delutto@unipi.

2 Processore, memoria e istruzioni

2.1 Processore

Il **processore** è in grado di fare operazioni piccole.

Il processore contiene i **registri**, memorie piccolissime e velocissime utilizzate per memorizzare dati temporanei legati alle istruzioni da eseguire. Ecco alcuni registri speciali:

- **Program Counter (PC)**: un registro speciale che conserva lo stato di esecuzione memorizzando l'indirizzo della prossima istruzione da eseguire.
- **Instruction Register (IR)**: contiene l'istruzione prelevata dalla memoria che deve essere eseguita.
- **Stack Pointer (SP)**: contiene la posizione attuale dello stack.

2.2 Memoria

Il processore è unito ad una **memoria**, costituita da **celle**, è vista come un vettore:

$$M[i] = k$$

Ogni cella ha un valore e la sua capacità è 1 parola (32 o 64 bit).

Una volta che il processore viene acceso, esso esegue:

```
while(true):  
    instr = getInstructionFrom(M)  
    decoded = decode(instr)  
    execute(decoded)  
    determineResults()  
    ...
```

Il processore quindi:

- Estrae istruzioni dalla memoria
- Le decodifica
- Le esegue
- Calcola i risultati

2.3 Tipi di istruzioni

Le istruzioni possono essere:

- Operative
 - Aritmetiche
 - * ADD
 - * SUB
 - * MUL

- * DIV
 - Logiche
 - * AND
 - * OR
 - * NOT
 - Shift / rotazione
 - * SHIFT
 - * ROT
 - ...
- Di Memoria
 - LOAD
 - * Restituisce un valore da un indirizzo:

LOAD val , addr
 - STORE
 - * Salva un valore in un indirizzo:

STORE val , addr
- di Salto
 - "Saltano" pezzi di codice, volendo anche secondo istruzioni, aggiornando il PC alla corrispondente istruzione.

```
inizio :  
  \dots  
  jmp fine :  
  —skipped code—  
fine :
```